

MODUL 13

Node.js, Express.js, MySQL (Pengelolaan Data Mahasiswa)



**Universitas
Telkom**

Disusun Oleh :

Nama : Ganes Gemi Putra

Kelas : SE-07-02

NIM : (2311104075)

Dosen : ARIF AMRULLOH S.Kom M.Kom

**PROGRAM STUDI S1 SOFTWARE ENGINEERING
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY
PURWOKERTO
2025**

1. Tujuan

Membangun aplikasi web sederhana untuk pengelolaan data mahasiswa menggunakan **Node.js**, **Express.js**, dan **MySQL**. Aplikasi menyediakan **RESTful API (CRUD)** serta dapat diakses melalui **browser** menggunakan halaman web sederhana berbasis **HTML + JavaScript**, dengan tambahan **CSS** untuk memperbaiki tampilan agar lebih rapi dan mudah digunakan.

2. Tools & Teknologi

- **Node.js + NPM** (runtime dan manajemen package)
- **Express.js** (framework backend untuk REST API)
- **MySQL** (database)
- **XAMPP/phpMyAdmin** (pengelolaan database)
- **Browser** (pengujian UI)
- **Postman** (opsional untuk uji API)
- **HTML + JavaScript (Fetch API)** (frontend)
- **CSS** (untuk styling tampilan UI)

3. Tahapan Pengerjaan (Sistematis)

3.1 Inisialisasi Proyek

1. Membuat folder proyek:

```
mkdir restfulAPI  
cd restfulAPI
```

2. Inisialisasi Node.js:

```
npm init -y
```

Perintah ini membuat file package.json sebagai konfigurasi proyek.

3. Instalasi dependency:

```
npm install express mysql
```

- express digunakan untuk membuat server dan routing API
- mysql digunakan untuk koneksi dan query ke database MySQL

3.2 Pembuatan Database

Membuat database dan tabel melalui phpMyAdmin (SQL):

```
CREATE DATABASE akademik;  
USE akademik;  
  
CREATE TABLE mahasiswa (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nama VARCHAR(100) NOT NULL,  
  nim VARCHAR(20) NOT NULL,  
  jurusan VARCHAR(50) NOT NULL,  
  email VARCHAR(100) NOT NULL  
);
```

3.3 Pembuatan Backend (Node.js + Express.js)

a) Koneksi Database (db.js)

File db.js digunakan untuk mengatur koneksi aplikasi ke MySQL (host, user, password, database).

b) Operasi CRUD (crud.js)

File crud.js berisi fungsi:

- createMahasiswa() → INSERT
- getAllMahasiswa() → SELECT
- updateMahasiswa() → UPDATE

- deleteMahasiswa() → DELETE
Fungsi-fungsi ini dipanggil dari route di app.js.

c) Routing REST API (app.js)

File app.js menjadi pusat server Express, mengatur endpoint REST API:

Method	Endpoint	Fungsi
POST	/mahasiswaCreate	Menambah data mahasiswa
GET	/mahasiswaGet	Mengambil semua data mahasiswa
PUT	/mahasiswaUpdate/:id	Mengubah data mahasiswa
DELETE	/mahasiswaDelete/:id	Menghapus data mahasiswa

Selain itu, aplikasi juga menggunakan:

```
app.use(express.static("public"));
```

Agar halaman frontend dapat diakses lewat browser.

3.4 Pembuatan Frontend (HTML + JavaScript)

Frontend disimpan dalam folder public/, terdiri dari:

- index.html → tampilan form input dan tabel daftar mahasiswa
- script.js → logic fetch API untuk memanggil REST API
- style.css → styling agar tampilan lebih rapi

a) Alur komunikasi Frontend–Backend

1. User input data dari form di browser
2. script.js mengirim request dengan fetch() ke API Express:
 - POST untuk tambah data
 - GET untuk menampilkan data
 - PUT untuk update data
 - DELETE untuk hapus data
3. Backend memproses request dan menjalankan query MySQL melalui crud.js
4. Hasil dikembalikan ke browser dan tabel diperbarui

4. Penjelasan Tambahan CSS (Agar Tampilan Rapi)

CSS digunakan untuk:

- Mengatur layout card, margin, padding agar tidak berantakan
- Membuat form input lebih rapi dan konsisten
- Membuat tabel lebih jelas dan nyaman dibaca (header sticky, border, spacing)
- Memberikan tampilan modern (tema gelap) agar mudah digunakan

Penggunaan CSS diperbolehkan karena masih termasuk teknologi dasar web (HTML, CSS, JS) dan tidak menggunakan framework berat seperti Bootstrap/React.

5. Cara Menjalankan Aplikasi

1. Pastikan MySQL (XAMPP) menyala
2. Jalankan server:

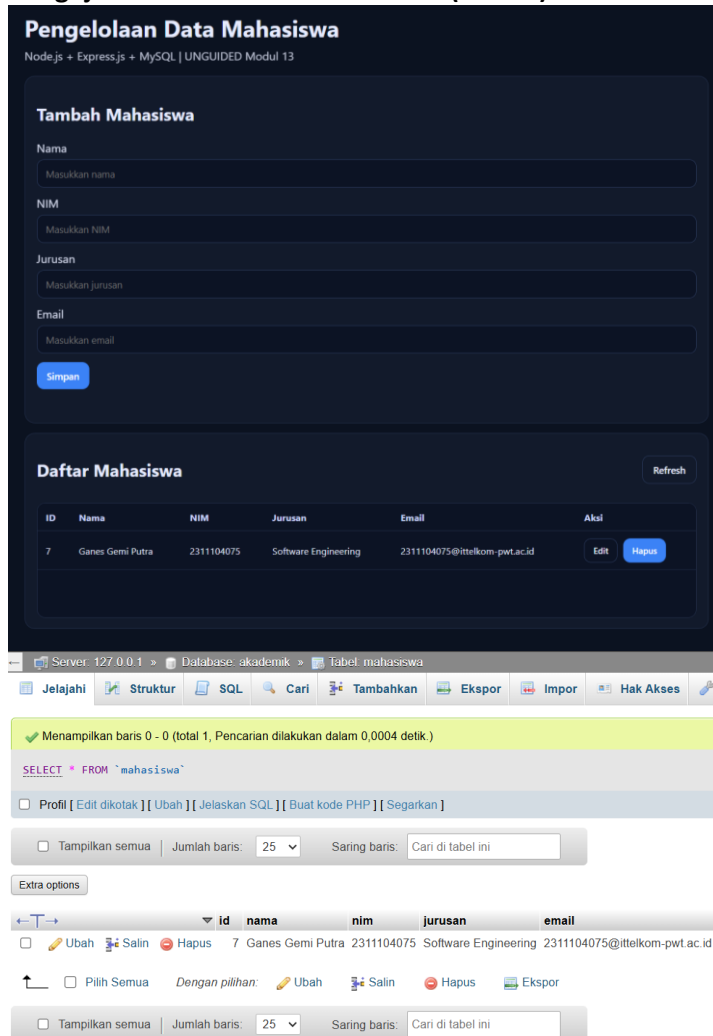

```
node app.js
```
3. Buka browser:


```
http://localhost:3000
```

6. Pengujian Sistem

Pengujian sistem dilakukan untuk memastikan seluruh fungsi CRUD pada aplikasi berjalan dengan baik dan sesuai dengan kebutuhan. Proses pengujian dilakukan melalui halaman web yang diakses menggunakan browser.

1. Pengujian Tambah Data Mahasiswa (Create)



Pengelolaan Data Mahasiswa
Node.js + Express.js + MySQL | UNGUIDED Modul 13

Tambah Mahasiswa

Nama
Masukkan nama

NIM
Masukkan NIM

Jurusan
Masukkan jurusan

Email
Masukkan email

Simpan

Daftar Mahasiswa Refresh

ID	Nama	NIM	Jurusan	Email	Aksi
7	Ganes Gemi Putra	2311104075	Software Engineering	2311104075@ittelkom-pwt.ac.id	Edit Hapus

Server: 127.0.0.1 > Database: akademik > Tabel: mahasiswa

Menampilkan baris 0 - 0 (total 1, Pencarian dilakukan dalam 0,0004 detik.)

`SELECT * FROM 'mahasiswa'`

☐ Profil ☐ Edit dikotak ☐ Ubah ☐ Jelaskan SQL ☐ Buat kode PHP ☐ Segarkan

☐ Tampilkan semua | Jumlah baris: 25 | Saring baris: Cari di tabel ini

Extra options

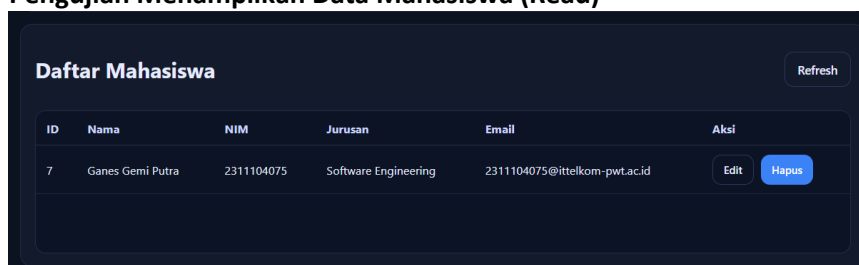
☐ Ubah ☐ Salin ☐ Hapus 7 Ganes Gemi Putra 2311104075 Software Engineering 2311104075@ittelkom-pwt.ac.id

☐ Pilih Semua Dengan pilihan: ☐ Ubah ☐ Salin ☐ Hapus ☐ Ekspor

☐ Tampilkan semua | Jumlah baris: 25 | Saring baris: Cari di tabel ini

Pengujian dilakukan dengan mengisi form tambah mahasiswa yang terdiri dari nama, NIM, jurusan, dan email. Setelah tombol **Simpan** ditekan, sistem mengirimkan permintaan HTTP **POST** ke server dan data mahasiswa berhasil disimpan ke dalam database MySQL.

2. Pengujian Menampilkan Data Mahasiswa (Read)



Daftar Mahasiswa Refresh

ID	Nama	NIM	Jurusan	Email	Aksi
7	Ganes Gemi Putra	2311104075	Software Engineering	2311104075@ittelkom-pwt.ac.id	Edit Hapus

Setelah data berhasil ditambahkan, sistem menampilkan data mahasiswa pada tabel daftar mahasiswa. Data diambil dari database menggunakan metode HTTP **GET** dan ditampilkan secara dinamis pada halaman web.

3. Pengujian Edit Data Mahasiswa (Update)

Pengelolaan Data Mahasiswa

Node.js + Express.js + MySQL | UNGUIDED Modul 13

Update Mahasiswa

Nama
Kanesuuuuu

NIM
2311104075

Jurusan
Software Engineering

Email
2311104075@ittelkom-pwt.ac.id

Update **Batal**

Daftar Mahasiswa

Refresh

ID	Nama	NIM	Jurusan	Email	Aksi
7	Kanesuuuuu	2311104075	Software Engineering	2311104075@ittelkom-pwt.ac.id	Edit Hapus

Server: 127.0.0.1 » Database: akademik » Tabel: mahasiswa

Jelajahi **Struktur** **SQL** **Cari** **Tambahkan** **Ekspor** **Impor** **Hak Akses**

Menampilkan baris 0 - 0 (total 1, Pencarian dilakukan dalam 0,0002 detik.)

`SELECT * FROM `mahasiswa``

☐ Profil ☐ Edit kotak ☐ Ubah ☐ Jelaskan SQL ☐ Buat kode PHP ☐ Segarkan

☐ Tampilkan semua | Jumlah baris: 25 | Saring baris: Cari di tabel ini

Extra options

	id	nama	nim	jurusan	email
☐ Ubah ☐ Salin ☐ Hapus	7	Kanesuuuuu	2311104075	Software Engineering	2311104075@ittelkom-pwt.ac.id

☐ Pilih Semua Dengan pilihan: ☐ Ubah ☐ Salin ☐ Hapus ☐ Ekspor

☐ Tampilkan semua | Jumlah baris: 25 | Saring baris: Cari di tabel ini

Pengujian dilakukan dengan menekan tombol **Edit** pada salah satu data mahasiswa. Data yang dipilih akan dimuat kembali ke dalam form, kemudian dilakukan perubahan dan disimpan menggunakan metode HTTP **PUT**. Data mahasiswa berhasil diperbarui di database.

4. Pengujian Hapus Data Mahasiswa (Delete)

localhost:3000 menyatakan

Yakin hapus data ID 7?

OK Batal

Pengelolaan Data Mahasiswa

Node.js + Express.js + MySQL | UNGUIDED Modul 13

Tambah Mahasiswa

Nama
Masukkan nama

NIM
Masukkan NIM

Jurusan
Masukkan jurusan

Email
Masukkan email

Simpan

Daftar Mahasiswa

Refresh

ID	Nama	NIM	Jurusan	Email	Aksi
Data kosong.					

Server: 127.0.0.1 » Database: akademik » Tabel: mahasiswa

Jelajahi Struktur SQL Cari Tambahkan Ekspor Im

MySQL memberikan hasil kosong (atau nol baris). (Pencarian dilakukan dalam 0,0002 detik.)

SELECT * FROM `mahasiswa`

☐ Profil [Edit dikotak] [Ubah] [Jelaskan SQL] [Buat kode PHP] [Segarkan]

id nama nim jurusan email

Operasi hasil kueri

Buat tampilan

Pengujian penghapusan data dilakukan dengan menekan tombol **Hapus** pada tabel mahasiswa. Sistem mengirimkan permintaan HTTP **DELETE** ke server dan data mahasiswa berhasil dihapus dari database.

7. Berikut HTML,Css,Javascript nya : indeks.html :

```
<!doctype html>
<html lang="id">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width,initial-scale=1" />
  <title>Data Mahasiswa - UNGUIDED Modul 13</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <div class="container">
```

```

<h1>Pengelolaan Data Mahasiswa</h1>
<p class="sub">Node.js + Express.js + MySQL | UNGUIDED Modul 13</p>

<div class="card">
  <h2 id="formTitle">Tambah Mahasiswa</h2>

  <form id="mhsForm">
    <input type="hidden" id="id" />
    <label>Nama</label>
    <input id="nama" placeholder="Masukkan nama" required />

    <label>NIM</label>
    <input id="nim" placeholder="Masukkan NIM" required />

    <label>Jurusan</label>
    <input id="jurusan" placeholder="Masukkan jurusan" required />

    <label>Email</label>
    <input id="email" placeholder="Masukkan email" required />

    <div class="row">
      <button type="submit" id="btnSubmit">Simpan</button>
      <button type="button" id="btnCancel" class="ghost hidden">Batal</button>
    </div>
  </form>
  <div id="msg" class="msg"></div>
</div>

<div class="card table-card">
  <div class="row-between">
    <h2>Daftar Mahasiswa</h2>
    <button id="btnRefresh" class="ghost">Refresh</button>
  </div>

  <div class="table-wrapper">
    <table>
      <thead>
        <tr>
          <th>ID</th><th>Nama</th><th>NIM</th><th>Jurusan</th><th>Email</th><th>Aksi</th>
        </tr>
      </thead>
      <tbody id="tbody">
        <tr><td colspan="6">Memuat data...</td></tr>
      </tbody>
    </table>
  </div>
</div>
</div>

<script src="script.js"></script>
</body>
</html>

```

style.css :

```

*{box-sizing:border-box;font-family:system-ui,Segoe UI,Arial}
body{margin:0;background:#0b1220;color:#e7ecff}
.container{max-width:980px;margin:32px auto;padding:0 16px}
h1{margin:0 0 6px}
.sub{margin:0 0 18px;opacity:.8}
.card{background:#121a2b;border:1px solid #24304b;border-

```

```

radius:14px;padding:16px;margin:14px 0}
label{display:block;margin-top:10px;margin-bottom:6px;opacity:.9}
input{width:100%;padding:10px 12px;border-radius:10px;border:1px solid
#2b3a5c;background:#0c1324;color:#e7ecff}
button{padding:10px 14px;border-
radius:10px;border:0;cursor:pointer;background:#3b82f6;color:white;font-weight:600}
button.ghost{background:transparent;border:1px solid #2b3a5c;color:#e7ecff}
.row{display:flex;gap:10px;margin-top:12px}
.row-between{display:flex;justify-content:space-between;align-items:center}
.actions{display:flex;gap:8px}
.msg{margin-top:10px;min-height:20px;opacity:.9}
.hidden{display:none}
.small{font-size:12px;opacity:.8}

/* ===== TABLE CARD ===== */
.table-card{
  padding: 14px 16px;
}

/* Wrapper agar tabel bisa kecil tapi tetap kelihatan */
.table-wrapper{
  margin-top: 10px;
  min-height: 160px;
  max-height: 240px;
  overflow: auto;
  border: 1px solid #24304b;
  border-radius: 12px;
  background: #0c1324;
}

/* Tabel rapi */
table{
  width: 100%;
  border-collapse: collapse;
}

thead th{
  position: sticky;
  top: 0;
  background: #0c1324; /* harus sama dengan wrapper */
  z-index: 2;
  text-align: left;
  font-weight: 700;
  font-size: 13px;
  color: #cbd5ff;
  border-bottom: 1px solid #24304b;
  padding: 10px 12px;
}

tbody td{
  padding: 10px 12px;
  font-size: 13px;
  border-bottom: 1px solid #1e2a44;
  color: #e7ecff;
  vertical-align: middle;
}

/* Stripe halus biar kebaca */
tbody tr:nth-child(even){
  background: rgba(255,255,255,0.02);
}

/* Empty state: "Data kosong." jelas */

```



```
tbody tr:only-child td[colspan]{
  text-align: center;
  color: #9aa4bf;
  font-style: italic;
  padding: 26px 12px;
  border-bottom: none;
}

/* Tombol aksi agar rapi */
.actions{
  display: flex;
  gap: 8px;
}
.actions button{
  padding: 8px 12px;
  border-radius: 10px;
  font-size: 12px;
}
```

Script.js :

```
// script.js - JavaScript untuk interaksi frontend dengan API
const API = ""; // karena frontend diserve dari server yang sama: http://localhost:3000

const tbody = document.getElementById("tbody");
const msg = document.getElementById("msg");

const form = document.getElementById("mhsForm");
const formTitle = document.getElementById("formTitle");
const btnSubmit = document.getElementById("btnSubmit");
const btnCancel = document.getElementById("btnCancel");
const btnRefresh = document.getElementById("btnRefresh");

const inputId = document.getElementById("id");
const inputNama = document.getElementById("nama");
const inputNim = document.getElementById("nim");
const inputJurusan = document.getElementById("jurusan");
const inputEmail = document.getElementById("email");

function setMessage(text, isError=false){
  msg.textContent = text;
  msg.style.color = isError ? "#ff8a8a" : "#9bffb3";
  if(!text) msg.style.color = "";
}

function resetForm(){
  inputId.value = "";
  formTitle.textContent = "Tambah Mahasiswa";
  btnSubmit.textContent = "Simpan";
  btnCancel.classList.add("hidden");
  form.reset();
}

async function loadData(){
  tbody.innerHTML = `<tr><td colspan="6">Memuat data...</td></tr>`;
  setMessage("");

  try{
    const res = await fetch(`${API}/mahasiswaGet`);
    if(!res.ok) throw new Error("Gagal mengambil data");
    const data = await res.json();
  }
```

```

if(!data.length){
  tbody.innerHTML = `<tr><td colspan="6">Data kosong.</td></tr>`;
  return;
}

tbody.innerHTML = data.map(row => `
  <tr>
    <td>${row.id}</td>
    <td>${row.nama}</td>
    <td>${row.nim}</td>
    <td>${row.jurusan}</td>
    <td>${row.email}</td>
    <td>
      <div class="actions">
        <button class="ghost" onclick='editRow(${JSON.stringify(row)})'>Edit</button>
        <button onclick="deleteRow(${row.id})">Hapus</button>
      </div>
    </td>
  </tr>
`).join("");

}catch(err){
  tbody.innerHTML = `<tr><td colspan="6">Error: ${err.message}</td></tr>`;
}
}

window.editRow = function(row){
  inputId.value = row.id;
  inputNama.value = row.nama;
  inputNim.value = row.nim;
  inputJurusan.value = row.jurusan;
  inputEmail.value = row.email;

  formTitle.textContent = "Update Mahasiswa";
  btnSubmit.textContent = "Update";
  btnCancel.classList.remove("hidden");
  setMessage("");
}

window.deleteRow = async function(id){
  if(!confirm(`Yakin hapus data ID ${id}?`)) return;

  try{
    const res = await fetch(`${API}/mahasiswaDelete/${id}`, { method: "DELETE" });
    const out = await res.json().catch(() => ({}));
    if(!res.ok) throw new Error(out.message || "Gagal menghapus");

    setMessage(out.message || "Berhasil menghapus");
    await loadData();
  }catch(err){
    setMessage(err.message, true);
  }
}

form.addEventListener("submit", async (e) => {
  e.preventDefault();
  setMessage("");

  const payload = {
    nama: inputNama.value.trim(),
    nim: inputNim.value.trim(),
    jurusan: inputJurusan.value.trim(),

```

```

    email: inputEmail.value.trim()
  };

  const id = inputId.value;

  try{
    if(!id){
      // CREATE
      const res = await fetch(`${API}/mahasiswaCreate`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(payload)
      });

      const out = await res.json().catch(() => ({}));
      if(!res.ok) throw new Error(out.message || "Gagal menambah data");

      setMessage(out.message || "Mahasiswa berhasil ditambahkan");
      resetForm();
      await loadData();
    }else{
      // UPDATE
      const res = await fetch(`${API}/mahasiswaUpdate/${id}`, {
        method: "PUT",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(payload)
      });

      const out = await res.json().catch(() => ({}));
      if(!res.ok) throw new Error(out.message || "Gagal update data");

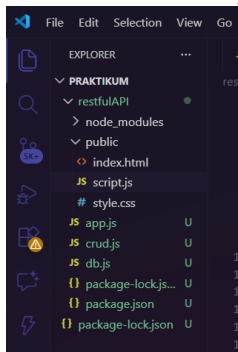
      setMessage(out.message || "Mahasiswa berhasil diupdate");
      resetForm();
      await loadData();
    }
  }catch(err){
    setMessage(err.message, true);
  }
});

btnCancel.addEventListener("click", resetForm);
btnRefresh.addEventListener("click", loadData);

loadData();

```

Isi folder dan file nya :





8. Hasil Implementasi

Aplikasi berhasil menampilkan:

- Form **Tambah Mahasiswa**
- Tabel **Daftar Mahasiswa** yang menampilkan data dari MySQL
- Tombol **Edit** dan **Hapus** yang terhubung ke API
- Tombol **Refresh** untuk memuat ulang data

9. Kesimpulan

Pada praktikum UNGUIDED Modul 13, berhasil dibuat aplikasi web sederhana pengelolaan data mahasiswa menggunakan Node.js, Express.js, dan MySQL. Sistem menyediakan RESTful API untuk operasi CRUD serta menyediakan halaman web berbasis HTML + JavaScript yang dapat diakses melalui browser. Penambahan CSS dilakukan untuk meningkatkan kerapihan tampilan dan kemudahan penggunaan tanpa mengubah konsep utama RESTful API.